

Presentation for use with the textbook **Data Structures and Algorithms in Java, 6th edition**, by M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, Wiley, 2014

Lists and Iterators



The java.util.List ADT

- The java.util.List interface includes the following methods:

`size()`: Returns the number of elements in the list.

`isEmpty()`: Returns a boolean indicating whether the list is empty.

`get(i)`: Returns the element of the list having index *i*; an error condition occurs if *i* is not in range $[0, \text{size}() - 1]$.

`set(i, e)`: Replaces the element at index *i* with *e*, and returns the old element that was replaced; an error condition occurs if *i* is not in range $[0, \text{size}() - 1]$.

`add(i, e)`: Inserts a new element *e* into the list so that it has index *i*, moving all subsequent elements one index later in the list; an error condition occurs if *i* is not in range $[0, \text{size}()]$.

`remove(i)`: Removes and returns the element at index *i*, moving all subsequent elements one index earlier in the list; an error condition occurs if *i* is not in range $[0, \text{size}() - 1]$.

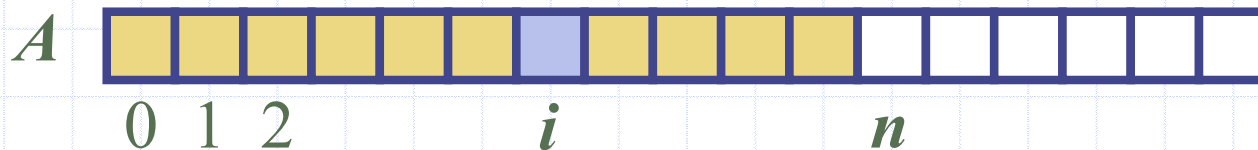
Example

- A sequence of List operations:

Method	Return Value	List Contents
add(0, A)	—	(A)
add(0, B)	—	(B, A)
get(1)	A	(B, A)
set(2, C)	“error”	(B, A)
add(2, C)	—	(B, A, C)
add(4, D)	“error”	(B, A, C)
remove(1)	A	(B, C)
add(1, D)	—	(B, D, C)
add(1, E)	—	(B, E, D, C)
get(4)	“error”	(B, E, D, C)
add(4, F)	—	(B, E, D, C, F)
set(2, G)	D	(B, E, G, C, F)
get(2)	G	(B, E, G, C, F)

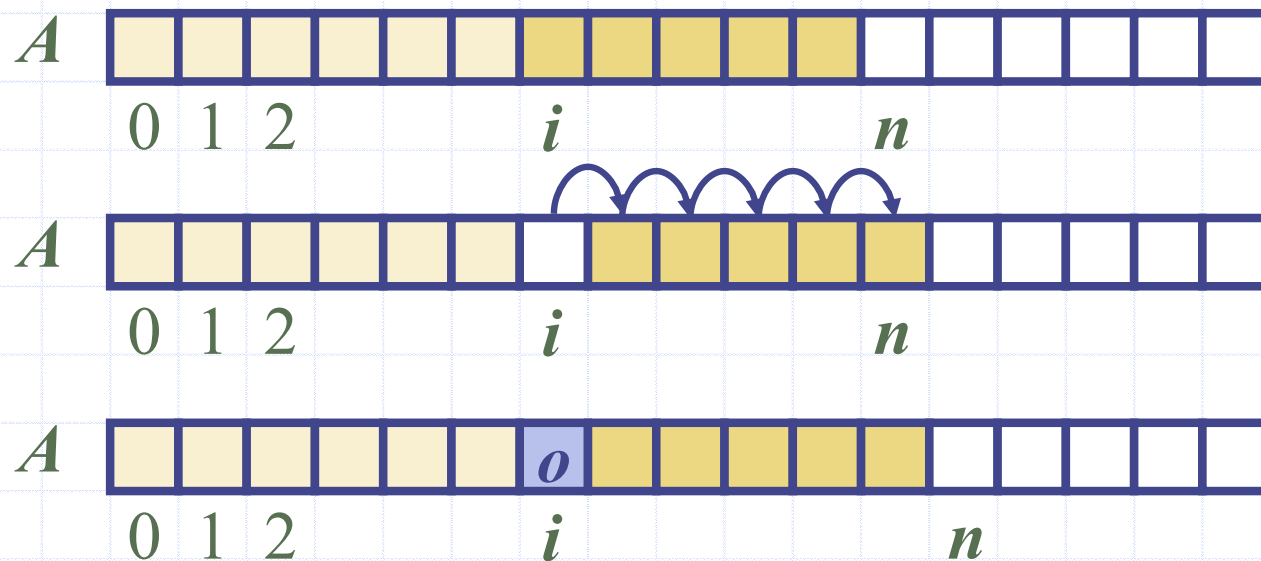
Array Lists

- An obvious choice for implementing the list ADT is to use an array, **A**, where **A[i]** stores (a reference to) the element with index **i**.
- With a representation based on an array **A**, the **get(i)** and **set(i, e)** methods are easy to implement by accessing **A[i]** (assuming **i** is a legitimate index).



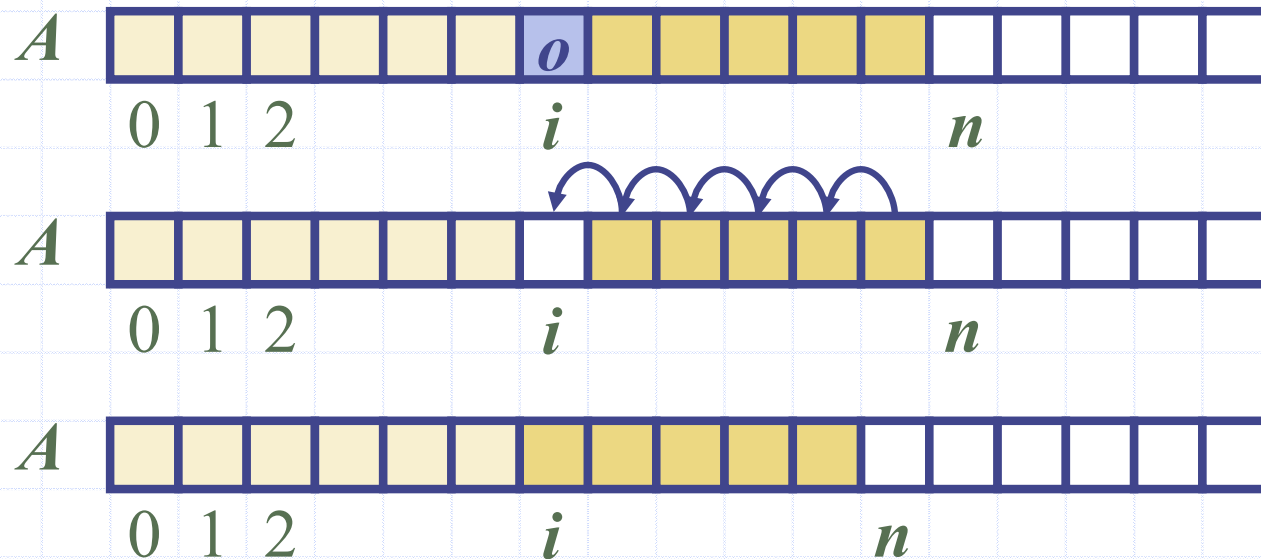
Insertion

- In an operation *add*(i, o), we need to make room for the new element by shifting forward the $n - i$ elements $A[i], \dots, A[n - 1]$
- In the worst case ($i = 0$), this takes $O(n)$ time



Element Removal

- In an operation *remove*(i), we need to fill the hole left by the removed element by shifting backward the $n - i - 1$ elements $A[i + 1], \dots, A[n - 1]$
- In the worst case ($i = 0$), this takes $O(n)$ time



Performance

- In an array-based implementation of a dynamic list:
 - The space used by the data structure is $O(n)$
 - Indexing the element at i takes $O(1)$ time
 - *add* and *remove* run in $O(n)$ time
- In an *add* operation, when the array is full, instead of throwing an exception, we can replace the array with a larger one ...

Java Implementation

```
11 // public methods
12 /** Returns the number of elements in the array list. */
13 public int size() { return size; }
14 /** Returns whether the array list is empty. */
15 public boolean isEmpty() { return size == 0; }
16 /** Returns (but does not remove) the element at index i. */
17 public E get(int i) throws IndexOutOfBoundsException {
18     checkIndex(i, size);
19     return data[i];
20 }
21 /** Replaces the element at index i with e, and returns the replaced element. */
22 public E set(int i, E e) throws IndexOutOfBoundsException {
23     checkIndex(i, size);
24     E temp = data[i];
25     data[i] = e;
26     return temp;
27 }
```

Java Implementation, 2

```
28  /** Inserts element e to be at index i, shifting all subsequent elements later. */
29  public void add(int i, E e) throws IndexOutOfBoundsException,
30                                     IllegalStateException {
31      checkIndex(i, size + 1);
32      if (size == data.length)          // not enough capacity
33          throw new IllegalStateException("Array is full");
34      for (int k=size-1; k >= i; k--)    // start by shifting rightmost
35          data[k+1] = data[k];
36      data[i] = e;                      // ready to place the new element
37      size++;
38  }
39  /** Removes/returns the element at index i, shifting subsequent elements earlier. */
40  public E remove(int i) throws IndexOutOfBoundsException {
41      checkIndex(i, size);
42      E temp = data[i];
43      for (int k=i; k < size-1; k++)    // shift elements to fill hole
44          data[k] = data[k+1];
45      data[size-1] = null;             // help garbage collection
46      size--;
47      return temp;
48  }
49  // utility method
50  /** Checks whether the given index is in the range [0, n-1]. */
51  protected void checkIndex(int i, int n) throws IndexOutOfBoundsException {
52      if (i < 0 || i >= n)
53          throw new IndexOutOfBoundsException("Illegal index: " + i);
54  }
55 }
```

Growable Array-based Array List

- Let **push(o)** be the operation that adds element **o** at the end of the list
- When the array is full, we replace the array with a larger one
- How large should the new array be?
 - **Incremental strategy**: increase the size by a constant c
 - **Doubling strategy**: double the size

```
Algorithm push(o)  
  if  $t = S.length - 1$  then  
     $A \leftarrow$  new array of  
      size ...  
    for  $i \leftarrow 0$  to  $n-1$  do  
       $A[i] \leftarrow S[i]$   
     $S \leftarrow A$   
     $n \leftarrow n + 1$   
     $S[n-1] \leftarrow o$ 
```

Comparison of the Strategies

- We compare the incremental strategy and the doubling strategy by analyzing the total time $T(n)$ needed to perform a series of n push operations
- We assume that we start with an empty list represented by a growable array of size 1
- We call **amortized time** of a push operation the average time taken by a push operation over the series of operations, i.e., $T(n)/n$

Incremental Strategy Analysis

- Over n push operations, we replace the array $k = n/c$ times, where c is a constant
- The total time $T(n)$ of a series of n push operations is proportional to

$$\begin{aligned}n + c + 2c + 3c + 4c + \dots + kc &= \\n + c(1 + 2 + 3 + \dots + k) &= \\n + ck(k + 1)/2\end{aligned}$$

- Since c is a constant, $T(n)$ is $O(n + k^2)$, i.e., $O(n^2)$
- Thus, the amortized time of a push operation is $O(n)$

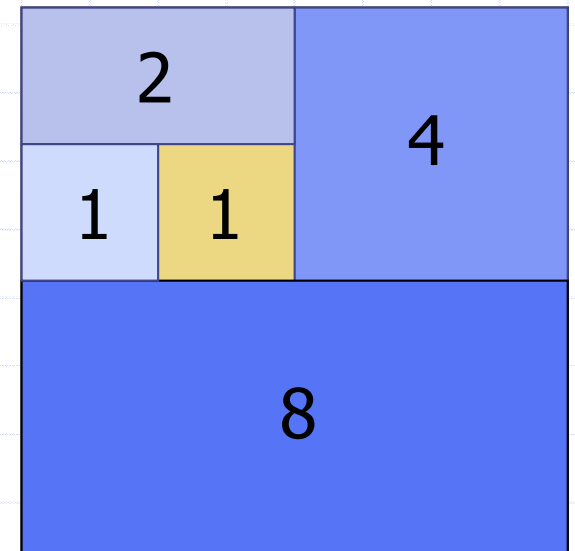
Doubling Strategy Analysis

- We replace the array $k = \log_2 n$ times
- The total time $T(n)$ of a series of n push operations is proportional to

$$\begin{aligned} n + 1 + 2 + 4 + 8 + \dots + 2^k &= \\ n + 2^{k+1} - 1 &= \\ 3n - 1 \end{aligned}$$

- $T(n)$ is $O(n)$
- The amortized time of a push operation is $O(1)$

geometric series



Positional Lists

- ❑ To provide for a general abstraction of a sequence of elements with the ability to identify the location of an element, we define a **positional list** ADT.
- ❑ A position acts as a marker or token within the broader positional list.
- ❑ A position p is unaffected by changes elsewhere in a list; the only way in which a position becomes invalid is if an explicit command is issued to delete it.
- ❑ A position instance is a simple object, supporting only the following method:
 - `P.getElement( )`: Return the element stored at position p .

Positional List ADT

□ Accessor methods:

`first()`: Returns the position of the first element of L (or null if empty).

`last()`: Returns the position of the last element of L (or null if empty).

`before( $p$ )`: Returns the position of L immediately before position p (or null if p is the first position).

`after( $p$ )`: Returns the position of L immediately after position p (or null if p is the last position).

`isEmpty()`: Returns true if list L does not contain any elements.

`size()`: Returns the number of elements in list L .

Positional List ADT, 2

□ Update methods:

`addFirst( $e$ )`: Inserts a new element e at the front of the list, returning the position of the new element.

`addLast( $e$ )`: Inserts a new element e at the back of the list, returning the position of the new element.

`addBefore( $p, e$ )`: Inserts a new element e in the list, just before position p , returning the position of the new element.

`addAfter( $p, e$ )`: Inserts a new element e in the list, just after position p , returning the position of the new element.

`set( $p, e$ )`: Replaces the element at position p with element e , returning the element formerly at position p .

`remove( $p$ )`: Removes and returns the element at position p in the list, invalidating the position.

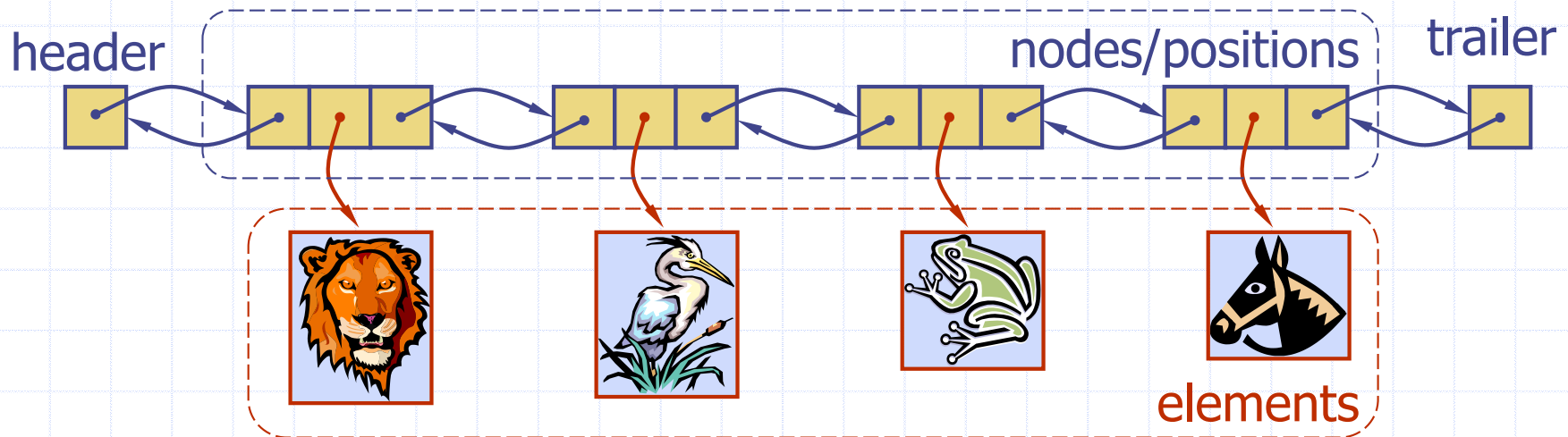
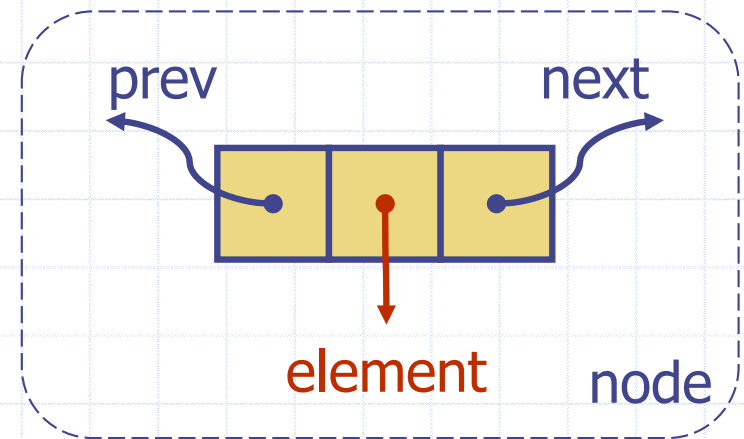
Example

- A sequence of Positional List operations:

Method	Return Value	List Contents
addLast(8)	p	$(8p)$
first()	p	$(8p)$
addAfter(p , 5)	q	$(8p, 5q)$
before(q)	p	$(8p, 5q)$
addBefore(q , 3)	r	$(8p, 3r, 5q)$
r .getElement()	3	$(8p, 3r, 5q)$
after(p)	r	$(8p, 3r, 5q)$
before(p)	null	$(8p, 3r, 5q)$
addFirst(9)	s	$(9s, 8p, 3r, 5q)$
remove(last())	5	$(9s, 8p, 3r)$
set(p , 7)	8	$(9s, 7p, 3r)$
remove(q)	“error”	$(9s, 7p, 3r)$

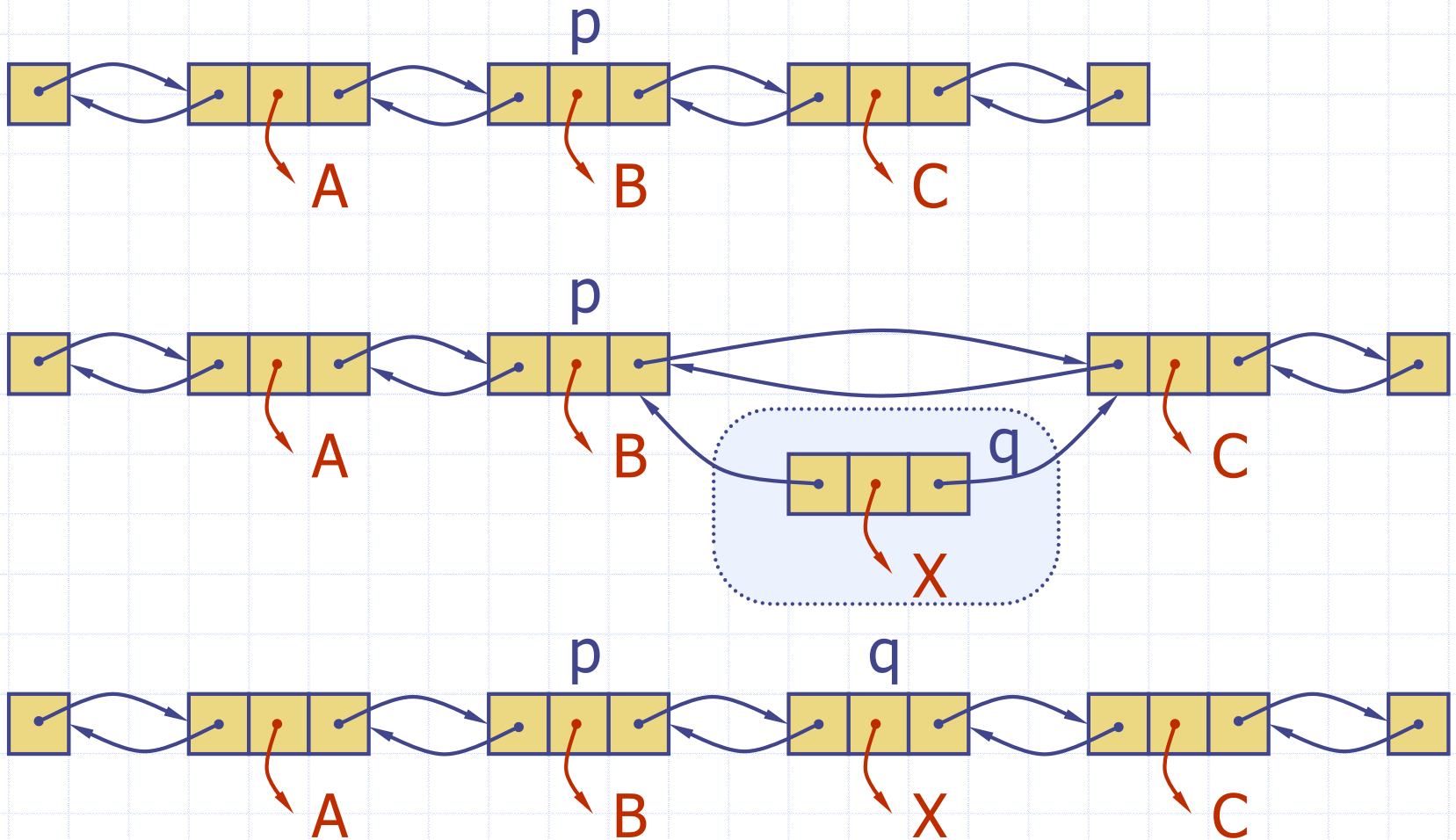
Positional List Implementation

- The most natural way to implement a positional list is with a doubly-linked list.



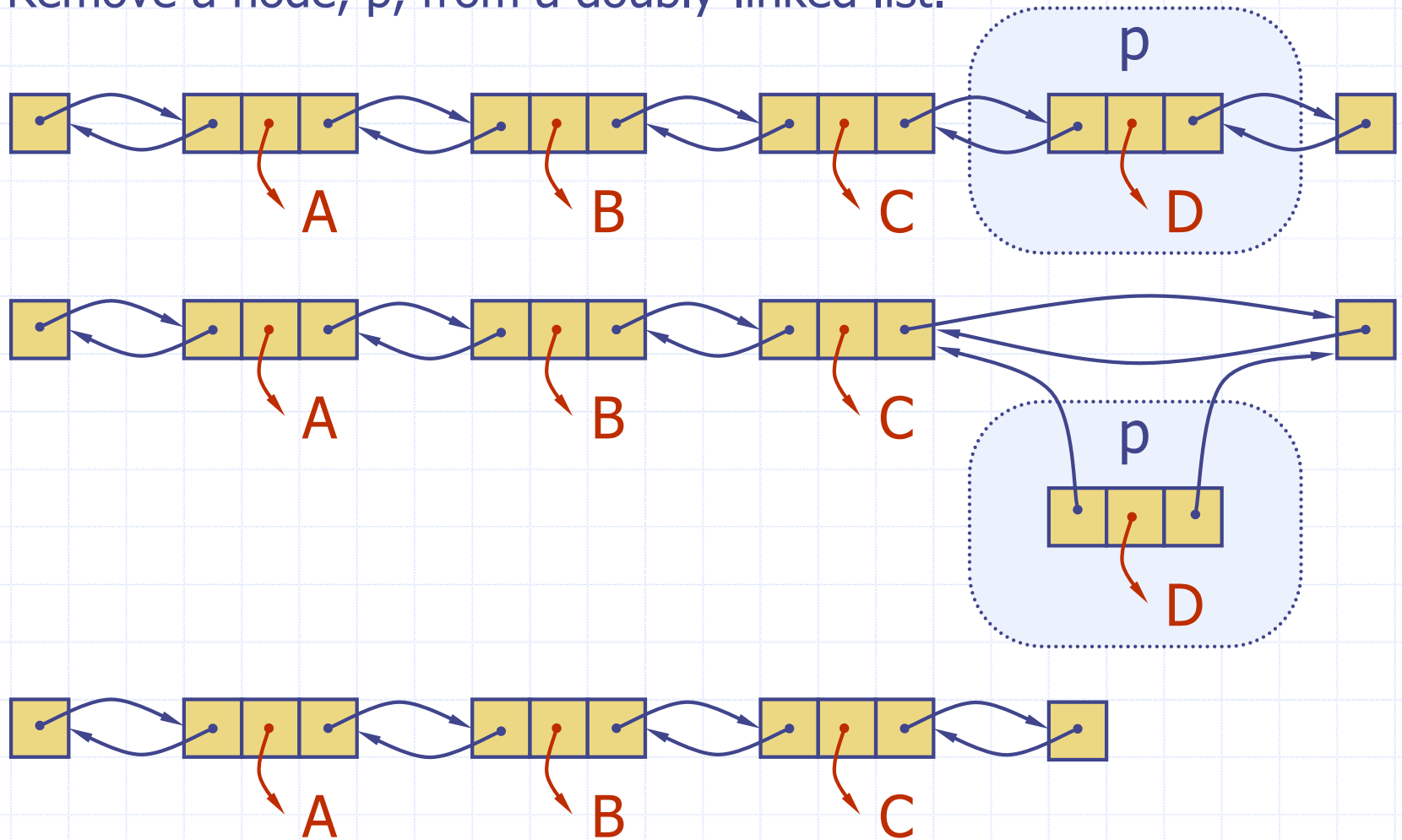
Insertion

- Insert a new node, q , between p and its successor.



Deletion

- Remove a node, p , from a doubly-linked list.



Iterators

- An iterator is a software design pattern that abstracts the process of scanning through a sequence of elements, one element at a time.

`hasNext()`: Returns true if there is at least one additional element in the sequence, and false otherwise.

`next()`: Returns the next element in the sequence.

The Iterable Interface

- ❑ Java defines a parameterized interface, named **Iterable**, that includes the following single method:
 - **iterator()**: Returns an iterator of the elements in the collection.
- ❑ An instance of a typical collection class in Java, such as an `ArrayList`, is iterable (but not itself an iterator); it produces an iterator for its collection as the return value of the **iterator()** method.
- ❑ Each call to **iterator()** returns a new iterator instance, thereby allowing multiple (even simultaneous) traversals of a collection.

The for-each Loop

- Java's Iterable class also plays a fundamental role in support of the "for-each" loop syntax:

```
for (ElementType variable : collection) {  
    loopBody                                // may refer to "variable"  
}
```

is equivalent to:

```
Iterator<ElementType> iter = collection.iterator();  
while (iter.hasNext()) {  
    ElementType variable = iter.next();  
    loopBody                                // may refer to "variable"  
}
```